# Overlapping Communication and Computation with ExaMPI's Strong Progress and Modern C++ Design

Derek Schafer
*University of Tennessee at Chattanooga*
Tennessee, USA
derek-schafer@utc.edu

Thomas Hines
*University of Tennessee at Chattanooga*
Tennessee, USA
Thomas-Hines01@utc.edu

Evan Drake Suggs
*University of Tennessee at Chattanooga*
Tennessee, USA
mdk988@mocs.utc.edu

Martin Rüfenacht
*Leibniz Supercomputing Centre*
Bavaria, Germany
martin.ruefenacht@lrz.de

Anthony Skjellum
*University of Tennessee at Chattanooga*
Tennessee, USA
Tony-Skjellum@utc.edu

***Abstract*—ExaMPI is a modern, C++17+ MPI implementation designed for modularity, extensibility, and understandability. In this work, we overview functionality new to ExaMPI since its initial release, including Libfabric-based network transport support. We also explain our rationale for why and how we choose to add new MPI features (and defer others). Lastly, we measured the latency of the aforementioned transports in ExaMPI and found that ExaMPI, while having slightly higher latency than other production MPIs, is competitive.**

**It is no longer uncommon to see MPI applications using extra MPI calls during non-blocking MPI operations to coax MPI's progress engine. Strong, asynchronous progress (aka application bypass) in MPI is instead based on the premise that an application asks MPI to perform a non-blocking communication in the background and MPI completes said communication without requiring any additional MPI calls from the application to advance the underlying transport. Strong progress often requires an additional background thread, but with the current trend in exascale computing, cores appear to be in excess. Indeed, for earlier MPI implementations that supported it well, strong progress enabled overlap and reduced time to completion for some MPI applications. However, enabling or adding strong progress to existing MPI implementations is not straightforward; changing such implementations is cumbersome, difficult, invasive, and time-consuming—a key motivation for our research MPI implementation, ExaMPI. Specifically, we tested the ability for ExaMPI's strong progress engine to enable overlap communication and computation, finding that considerable overlap is achieved without needing additional MPI "helper" calls such as `MPI_Test`.**

***Index Terms*—MPI, middleware architecture, parallel programming models, overlap, performance portability**

## I. Introduction

The complexity of leading open-source implementations of MPI is daunting when it comes to experimentation and modification with new and different concepts for MPI-4.1, MPI-5, and/or other research experiments. Production open source MPIs have successfully focused on completeness of coverage, correctness, compliance, and, of course, middleware portability and performance. But they typically leveraged software architectures rooted in legacy implementations of MPI-1 or earlier message passing systems, where assumptions were made based on then-extant architectures, processor resources, assumptions of intra-node concurrency, and performance levels. Production open-source MPIs possess complex internal architectures, layers, and global state, and cross-cutting issues can arise when trying to experiment. ExaMPI was introduced to help alleviate these issues by providing an experimental research implementation of the MPI Standard [1]. ExaMPI was designed to be a modular, C++17+ MPI implementation that is easy to learn and modify. These features helped the authors explore the usage of multiple, interchangeable progress models (including strong progress) inside an MPI implementation, multiple fault tolerance models, and multi-language support in MPI. This paper focuses on the first area, but the new features explored in this paper may also prove useful to the other areas mentioned.

Since our initial release of ExaMPI [1], we have continued to enhance the features provided by our MPI implementation. Besides general improvements, we increased the number of MPI procedures supported by ExaMPI, as well as added support for more network transports. These additions were mainly driven by our desire to test strong progress in real applications. In this paper, we tested ExaMPI's latency and its ability to achieve good overlap with both its strong and weak progress engines.

The remainder of the paper is organized as follows: Section II explains the motivations and original design principles for our MPI implementation, with a specific focus on strong progress in MPI implementations. Section III presents the new features and changes since the first release of ExaMPI. Section IV focuses specifically on the current function coverage of ExaMPI. We then demonstrate and discuss latency and overlap performance results in Section V. Future work and projects with ExaMPI and discussed in Section VI before concluding the paper in Section VII.

## II. Background

This section discusses the ability of other MPI implementations to handle strong progress as well as recaps where

2021 Workshop on Exascale MPI (ExaMPI) | 978-1-6654-1108-0/21/$31.00 ©2021 IEEE | DOI: 10.1109/EXAMPI54564.2021.00008

978-1-6654-1108-0/21/$31.00 ©2021 IEEE
DOI 10.1109/ExaMPI54564.2021.00008



Authorized licensed use limited to: Technische Universitaet Muenchen. Downloaded on May 01,2026 at 17:35:55 UTC from IEEE Xplore. Restrictions apply.

ExaMPI stood prior to this work.

### A. Quick Summary of Other MPI Implementations

Support for strong, asynchronous progress (application bypass) from modern MPI implementations remains sparse and/or limited. Today, there are two main open-source MPI implementations: MPICH [2] and Open MPI [3]. We have verified anecdotally that by itself, Open MPI cannot support strong progress, although it can leverage progress deriving from underlying transports[1]. MPICH's documentation makes no mention of strong progress [2]. Commercial and free derivative products of these open implementations, such as Cray MPI [4], Intel MPI [5], IBM Spectrum MPI [6], and MVAPICH [7] have also been successful, but also do not offer overt strong progress support. Intel MPI has an option that lets one set per-communicator progress threads, but this is not really strong progress, as individual messages on a given communicator are not guaranteed to be progressed asynchronously without assistance from the user program; what is more, the MPI progress rule requires cross-communicator progress whenever any completing call is made [8]. Ruhela et al. report that they recently added a feature to MVAPICH2-X enabling a user to specifically request a separate, asynchronous progress thread to be used [9]. This additional thread uses `MPI_Test` to help progress any large outstanding messages, and uses conditional variables to help avoid contention between the user thread and their progress thread.

### B. Quick Summary of ExaMPI

In several research areas of interest to the authors, we sought to explore different methodologies for implementing MPI, including new modes of strong progress, fault tolerant concepts, and extensions to the standard that require highly effective progress[2]. However, the complexity of producing significant improvements, modifications, and/or changes in design to these large-scale MPIs is a daunting task. With several hundred thousand lines of code present in both middleware products, deep experimentation with, or changes to, MPI is prohibitive, except in device drivers and incremental APIs. For instance, changing the modes of progress or the modes of completion of MPI implementations is a tall order, as is managing their ability to cope with internal concurrency or state. By supporting a sound design and allowing functionality to be added systematically over time, we provide incremental ability to run practical codes while reducing the total amount of MPI middleware by orders of magnitude. Furthermore, with this sound, first-principles design, ExaMPI has little dead code, or the technical debt associated with assumptions about node concurrency or progress made in the 1990's.

[1]Personal communication with Dr. Howard Pritchard, LANL, a lead Open MPI developer, August 2021.

[2]As a reminder, we use "strong" to mean that the progress is independent with separate progress threads from the user threads and the notification of completion is blocking. When a user thread waits on a request, the user thread is unscheduled until the request is complete. We recognize that applications with many short messages work better with strong progress paired with polling completion, but this remains for a future communication.

As mentioned in our prior work, the goal of ExaMPI is not to replace open-source, commercial, or derivative MPI implementations, but rather to base ExaMPI on several design principles that will enhance and simplify researchers' ability to explore new functionality within MPI. These principles are summarized below:

- Enable rapid prototyping of new algorithms, concepts and features
- Identify and elucidate opportunities to improve performance and understanding of MPI at-large
- Support the research interests and experiments of the authors (and others), such as effective overlap of communication and computation.

In the previous work, ExaMPI only supported a few collectives and a subset of the MPI 3.1 API. Since then we have expanded the capabilities and coverage of ExaMPI to support more functionality, which will be discussed more in Section III and Section IV.

## III. Design Updates

The current version of ExaMPI features several general improvements to the library. We have refactored the code base to fully be C++-based internally, and as such, ExaMPI now no longer depends on `mpi.h` internally and is free to make its own definitions for various objects. This refactoring allows ExaMPI to support more C++17+ features internally, and potentially expose these features to a future C++ interface. One of the key benefits is that now all key MPI objects take advantage of the smart pointers provided by C++ behind the scenes. These smart pointers allow for easier reference counting and memory management of objects. Additionally, we have slowly added more functions from the MPI Standard to ExaMPI; see Section IV for further discussion. While most of the original design requirements are still followed, some alterations to the core code has required alterations of these requirements, which is what the remainder of this section will focus on.

As mentioned in our prior work, Figure 1 shows the different modes of progress and classification. Since the initial release of ExaMPI, a weak progress engine option has also been added to ExaMPI. This progress engine, as the diagram and name imply, only makes progress when the application calls MPI. More specifically, our weak progress engine only makes progress when progress-related functions are called, such as blocking communication calls, `MPI_Test`-based calls, and `MPI_Wait`-based calls[3]. No additional progress is attempted by other MPI function calls; if an MPI function does not perform communication, it does not attempt to also perform progress.

Additionally, we have altered our design on handling interfaces to ExaMPI. Previously, the flow of any given MPI call was as follows:

[3]Technically, `MPI_Request_get_status` should be included in this list, but the progress rules for this function are being explored in the MPI Forum at this time.



Authorized licensed use limited to: Technische Universitaet Muenchen. Downloaded on May 01,2026 at 17:35:55 UTC from IEEE Xplore. Restrictions apply.

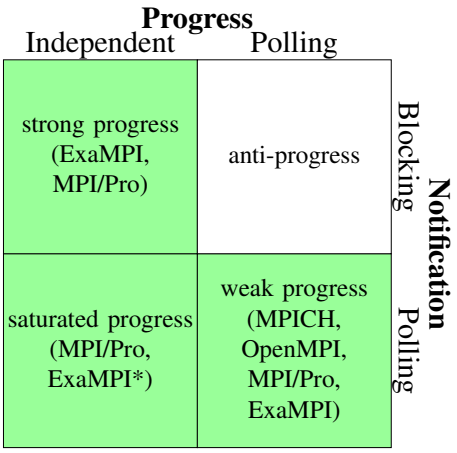



Fig. 1. Dimitrov's Progress and Notification Classification Diagram adopted from [10]; * Forthcoming mode in ExaMPI

- Call PMPI Interface
- Obtain the root `Universe` object and call corresponding function
- Corresponding function calls into the internals of ExaMPI

Instead of the PMPI layer having to go through a root `Universe` to find the underlying C++ implementation of the function, the PMPI layers now go directly to the related underlying C++ objects to effect their behavior; the new design allows us to skip the second step. This is shown in Figure 2. Here, we introduce a "translation" layer that helps the non-C++ interfaces convert the MPI Objects into their underlying C++ object. Here, the `Powertool` object is the new version of the `Universe` object, provided access to global resources for any API that requires them (such as a global request pool to allocate new `MPI_Requests` from). The `Toolbox` box represents a namespace of static helper functions designed to reduce code duplication. Language-conversion functions are examples of functions found in this namespace.

We have also begun work on Fortran and C++ interfaces. The Fortran interface mirrors the MPI Standard and uses the C interface underneath. On the other hand, the C++ interface is more exploratory since the C++ MPI interface was deleted from MPI years ago[4].

ExaMPI originally supported a UDP-based network transport and a TCP-based transport. UDP was introduced to explore state reduction for fault-tolerant computing. However, we have dropped support for the UDP-based transport, since we have found it to be unreliable (as was expected with this protocol; in the future we may revisit this transport with an effort to make it sufficiently reliable). Since that release, we have also added a Libfabric-based network transport to ExaMPI, enabling InfiniBand and other network support.

One of the original core design principles of ExaMPI was to make all communications persistent by default underneath,

[4]There are current efforts by the MPI Forum Languages Working Group to investigate bringing more language interfaces into the MPI Standard.

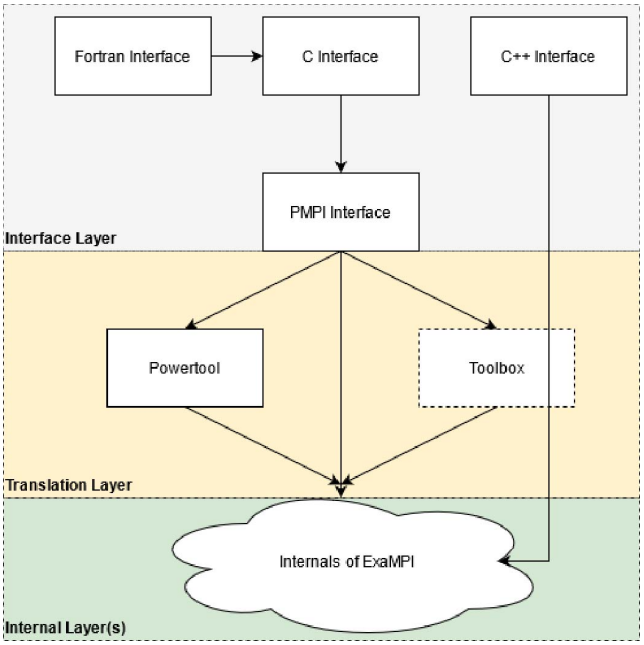



Fig. 2. New interface layer design

even if the user were to call a blocking MPI call. We have continued to support this principle as we extended the number of collectives supported by ExaMPI. Drawing inspiration from work done in [11], we have designed a framework for all of our collective algorithms to use that supports variations on the actual communication pattern. All collective algorithms build a schedule when they are first created, and when asked to progress by the progress engine, they go through their schedule and perform communications, memory management, and/or reduction operations as dictated by their schedule. Once done, these algorithms can be asked to reset and re-use the same schedule.

As we have continued to grow and experiment with new functions, the need for a testing framework has become apparent. Using the testing suite provided by the CMake build tool, we have added primitive MPI applications that help make sure that at least, the MPI APIs ExaMPI provides work as intended. In the future, we aim to have more thorough unit testing, but testing all cases and scenarios for all internal objects has proved to be difficult, especially with network-based objects.

Lastly, the runtime environment for ExaMPI has remained largely unchanged, maintaining the original design for the dæmons. Since our initial release, the dæmons have only undergone incremental updates and bug fixes, plus an update to Python 3.

## IV. Coverage

Coverage is an important aspect for an MPI implementation; without full coverage of the MPI Standard an implementation is not compliant. All aspects of the MPI Standard are a part



Authorized licensed use limited to: Technische Universitaet Muenchen. Downloaded on May 01,2026 at 17:35:55 UTC from IEEE Xplore. Restrictions apply.

of the coverage of an implementation, whether it is constants, mechanisms, or procedures. ExaMPI does not intend to be entirely compliant to the MPI Standard and therefore a choice must be made about what parts of the MPI Standard to implement. The choice of functionality is done by using prior surveys to determine which subsets of the MPI Standard are used and then to invest software development resources into those parts. This naturally leads to support of many, but not all applications, and when a subset is required by an application then it can be implemented.

Currently, 114 procedures are supported within ExaMPI of the 481 MPI procedures defined in the standard. This is a 24% coverage of the MPI Standard. This does not account for different languages expressions or variants such as the embiggened versions of communication procedures.

With the existing communication functionality of ExaMPI, we cover eight of the fourteen ECP Proxy Applications [12] surveyed in Sultana et al [13]. All point-to-point and regular collective functionality is supported for all ECP Proxy Applications, but the variable collectives, such as `MPI_Alltoallv`, are the primary reason for not supporting almost half the proxy applications. In addition to the communication, datatype and communicator functionality is the reason for two additional proxy applications to not be supported.

For the application survey by Laguna et al [14] ExaMPI supports 27% of all applications surveyed, 30 of 110 applications surveyed. If we accept a threshold of 80% of procedures used by an application being supported in ExaMPI, then the support applications jumps to 69%. This threshold allows us to gauge the amount of additional work required to achieve a determined level of MPI support. In similarity to to the proxy applications, variable collectives are the primary reason for failing to support an application. This is evidenced by 37 applications requiring `MPI_Allgatherv`, 30 requiring `MPI_Gatherv`, and 29 requiring `MPI_Alltoallv`. Table I shows the functions which require an implementation, ignoring one-sided and file operations, to achieved the stated number of applications to support per procedure.

TABLE I
THE MOST USED PROCEDURES IN THE MPI STANDARD (ACCORDING TO LAGUNA ET AL [14]) WHICH ARE NOT SUPPORTED BY EXAMPI.

| | |
|---|---|
| MPI_ALLGATHERV | 37 |
| MPI_GATHERV | 30 |
| MPI_ALLTOALLV | 29 |
| MPI_SCAN | 25 |
| MPI_ERROR_STRING | 21 |
| MPI_TYPE_CREATE_STRUCT | 19 |
| MPI_OP_CREATE | 17 |
| MPI_SCATTERV | 17 |
| MPI_IPROBE | 16 |
| MPI_GET_ADDRESS | 15 |

However, support for v/w-collectives mentioned above is coming in the near future. This will considerably advance the ability to support aforementioned applications, and many other applications not surveyed.

ExaMPI currently does not support any of the one-sided, file operations, or tools procedures from the MPI Standard. These procedures are not supported due to a combination of lack of usage and an ongoing effort to fully understand the semantics behind some of the procedures. Eventually, support for these procedures will be introduced into ExaMPI, but there are no immediate plans to support these functions.

## V. RESULTS

To verify and compare our implementation with others, two tests were created: a ping-pong latency test and an overlap test. These tests were carried out on the Epyc cluster at the University of Tennessee at Chattanooga, running CentOS Version 7. The Epyc system has 16 nodes, with each node using two AMD EPYC 7662 processors for a total of 128 cores per node. ExaMPI was installed along with Open MPI 4.1.0, MPICH 3.4.2, and Intel MPI 2021.2.0 on the cluster, and both tests were run using these release versions. All files were compiled with the gcc version 10.2.0 compiler on the system. The overlap test utilized the OpenBLAS library for its calculations [15]. Each test was run with two nodes and one process per node, with each process performing in single threaded mode for both MPI (`MPI_THREAD_SINGLE`) and OpenMP (`OMP_NUM_THREADS`=1).

### A. Ping-Pong Test

The ping-pong test is designed to directly test the messaging throughput by sending messages of sizes from one byte to one gigabyte and recording the time from start to end. The results of the ping-pong test with the various implementations are shown in Figure 3.

ExaMPI was tested using the strong and weak progress models shown in Figure 1, and each model was run using both TCP and InfiniBand transports.

The results of the production MPI implementations (Open MPI, Intel MPI and MPICH) are all clustered together at the top of the graph. Following them are the weak progress InfiniBand ExaMPI, trailing only slightly behind, and strong progress InfiniBand ExaMPI tests, initially trailing about an order of magnitude behind at smaller message sizes. Finally, the TCP results are around two orders of magnitude slower than those using InfiniBand. When using the TCP network transport, both progress modes recorded similar times, though weak progress achieved a marginally faster transfer rate.

For the InfiniBand compatible implementations, the transfer rate plateaus close to 11.5 gibibytes per second after the message size reaches around 100,000 bytes. The network bandwidth is 100 Gb/s or 11.6 GiB/s. Similarly, after 100,000 bytes ExaMPI's TCP transport plateau around 0.1 gibibytes per second.

A zoomed-in graph of the larger message sizes from Figure 3 is shown in Figure 4. Here, ExaMPI in weak progress mode manages to eke out a slightly faster transfer rate, and is able to hold it for longer than the other MPI implementations. The strong progress mode of ExaMPI features some noise in its pattern, but continues to mirror the weak progress results



Authorized licensed use limited to: Technische Universitaet Muenchen. Downloaded on May 01,2026 at 17:35:55 UTC from IEEE Xplore. Restrictions apply.

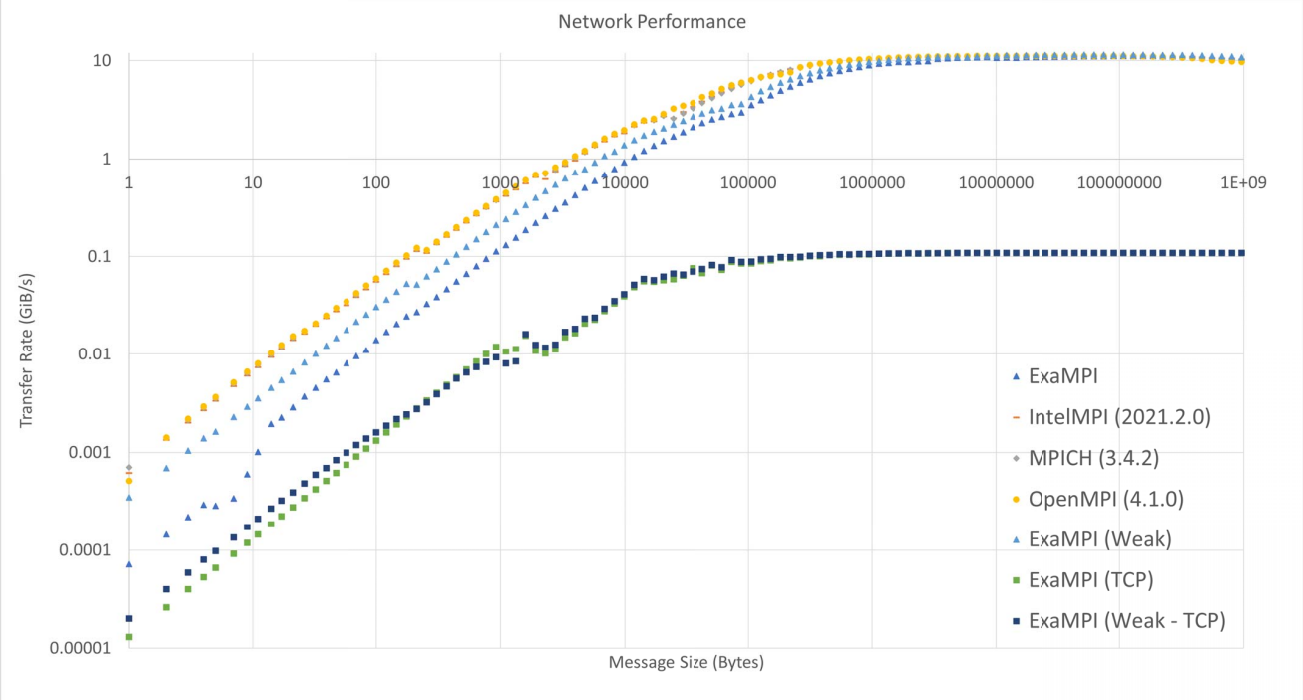



Fig. 3. Results for the ping-pong test, featuring three other MPI implementations and four builds of ExaMPI. Higher numbers are better, and the network limitation for the InfiniBand network is around 11.6 GiB/s.

from before with a slightly slower transfer rate. The other MPI implementations are all close to one another, with Intel MPI having a slower decline at the largest message sizes, though exploring the rationale for the decay is left for future work. Data points from the TCP Transports for ExaMPI have been omitted as there is little variance across their plateau after message size of one million bytes.

### B. Overlap Test

The overlap test is designed to assess how much communication and computation overlap an MPI implementation can achieve. The communication is a single send of 512 MiB. The computation is six calls to a single threaded DGEMM (Double GEneral Matrix Multiplication) designed to take the same amount of time as the communication. This means that full overlap should result in half the time of the compute and communication done sequentially. In this test, the first run for each test was omitted as a "warm-up" run. Each test and implementation ran 100 times.

As shown in Figure 5, there are five primary modes used by the test, as explained below:

- **Blocking (completing)** mode utilizes the traditional blocking MPI point-to-point calls and serves as a baseline statistic. It is usually the slowest mode and does not make any calls to `MPI_Test`.
- **No Tests** replaces the blocking MPI calls with their non-blocking equivalent, and puts an `MPI_Wait` at the end of all computation. The wait is included in the measured times. If an MPI implementation features weak progress, we expect these results to be similar to the blocking times. If it features strong progress, we expect to see some overlap.
- **One Test** adds a single call to `MPI_Test` halfway through the computations (after three DGEMM calculations). For implementations with weak progress, we expect the results to shows signs of some overlap.
- **Some Tests** puts in two `MPI_Test` calls. The first is inserted after two DGEMM calculations. The second is inserted after the fourth DGEMM calculation. Weak progress MPI implementations should continue to see increased overlap.
- **All Tests** puts in five `MPI_Test` calls, one after the first five DGEMM calculations. There is not a sixth test after the sixth DGEMM because of the MPI_Wait after all computations that would perform the same completion steps, if required at this point. For implementations with weak progress, we expect to see the greatest amount of overlap here.

The results of these tests are shown in Figure 5.

The condition are the same as in the ping-pong test and Figure 3, but without any runs using ExaMPI's TCP transport, as this experiment is designed to highlight how strong and weak progress affect performance. Unless otherwise noted, we do not draw any conclusions from implementations that feature marginally different times in the same test category. For example, even though we numerically recorded a "slower"



Authorized licensed use limited to: Technische Universitaet Muenchen. Downloaded on May 01,2026 at 17:35:55 UTC from IEEE Xplore. Restrictions apply.

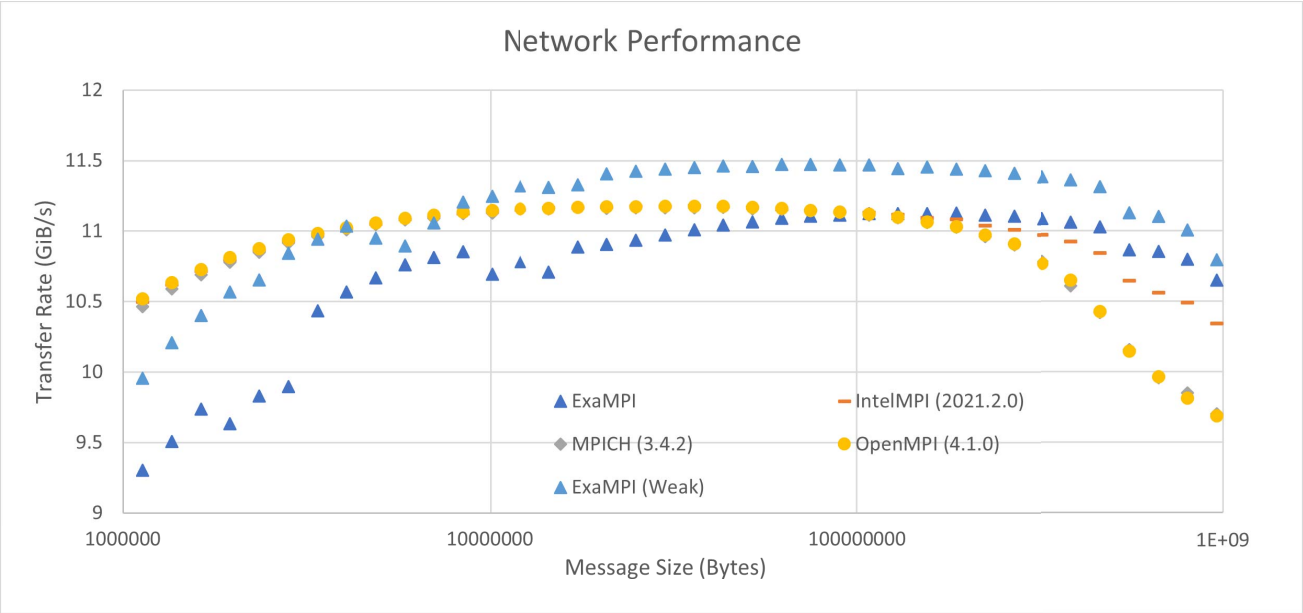



Fig. 4. A zoomed in version of the larger message sizes from Figure 3. Again, higher is better, and the network limitation for the InfiniBand network is around 11.6 GiB/s.

time for MPICH in the non-blocking category, we do not offer any rationale as to why. This section focuses only on noticeable differences between implementations in a given category (such as ExaMPI's strong progress performance in the "no tests" category) or the difference between two different categories.

As expected, the baseline blocking version of the test has the slowest overall time for all five implementations when compared to the other four versions of the experiment. When switching to the non-blocking version, OpenMPI, MPICH, and Intel MPI show no differences in average time of the experiment. However, ExaMPI immediately shows a noticeable time improvement, and manages to achieve roughly half the time of the blocking version, resulting in significant overlap.

When adding `MPI_Test` calls into the experiments, Open MPI, MPICH, Intel MPI, and ExaMPI in weak progress mode begin to achieve faster completion times, and thus record measurable overlap in the results. When one call to `MPI_Test` is introduced, the production MPI implementations are able to reduce the average time by roughly 25%. Adding more tests decreases the total run times. However, at the highest frequency of `MPI_Test` calls, the production MPI implementations do not achieve as much overlap as ExaMPI in strong progress mode with no `MPI_Tests`. ExaMPI's strong progress results remain unchanged no matter the frequency of `MPI_Test` calls.

ExaMPI's weak progress mode appears to follow the same pattern as the production MPI implementations: adding `MPI_Test` calls increases overlap. However, ExaMPI in this mode appears un-optimized, as many calls to MPI_Test were needed before consistent improvements to completion time were recorded.

## VI. Projects with ExaMPI

This section discusses all of the current research areas that are being exploring with ExaMPI.

### A. Past Projects

As noted above, the inspiration for the collective algorithms in ExaMPI is drawn from previous work on user-level schedules [11]. In this work, applications are provided access to an MPI extension that allows them to build out and describe their own communication pattern out of persistent `MPI_Request` objects. The authors are actively working on integration of this initial exploration with the new collective algorithm design in ExaMPI.

In the debut of ExaMPI, one of the key objectives was supporting fault tolerance. Specifically, we have been investigating FA-MPI and MPI Stages [16], [17]. Integration of the former is still ongoing, while the latter model continues to contribute new ideas to the area [18], [19].

In Herschberg et al., ExaMPI was used to explore the concept of using MPI Sessions to directly create topological connections [20]. These specific topological connections were combined with specifically crafted communication algorithms to test performance and initialization times.

### B. Ongoing Projects

From the work started by Herschberg et al., work has also began to fully support the MPI Sessions model. The intent of this work is to fully support the Sessions model as the default



Authorized licensed use limited to: Technische Universitaet Muenchen. Downloaded on May 01,2026 at 17:35:55 UTC from IEEE Xplore. Restrictions apply.

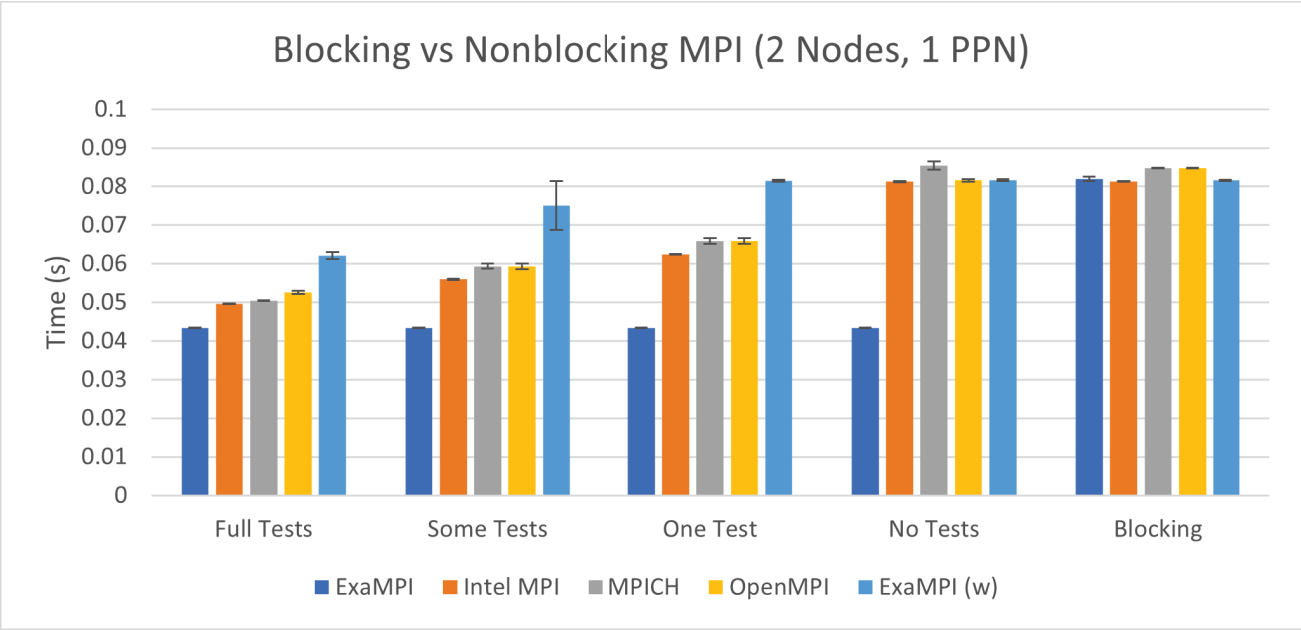



Fig. 5. Achieved overlap of communication and computation (lower is better). Inserting `MPI_Tests` helps MPICH [2], Open MPI [3], and Intel MPI [5]. ExaMPI [1] achieves best overlap without any `MPI_Tests`. The blocking category is the baseline for comparison. This test moves a 512 MiB message. Each category has the same order as the legend.

model for ExaMPI, and have the World model be a special case of the Sessions model[5].

Comprehensive research on FPGA-based offload of MPI collective operations, datatypes, and other protocols (NIC and/or switch optimizations) have been undertaken by the research group of Dr. Martin Herbordt at Boston University, in collaboration with some of the authors of this paper, and many others (e.g., [22]–[29]). In these efforts, ExaMPI is now the MPI implementation for the FPGA-offload work.

Some of the authors are also working on integrating the partitioned communication layered library created by Worley et al. into ExaMPI [30]. Lastly, as noted in Figure 2, we are currently investigating a Fortran 2018 interface and a new C++ interface.

### C. Future Work

Several outstanding problems confront MPI at present. GPU interaction with MPI is still limited, and the intersection with partitioned communication is of interest to the authors. Finding tractable components of fault tolerance is also another area of interest. We want to use ExaMPI to help explore these problems and derive reasonable solutions that can then be taken back to the larger MPI community as well as to accelerator vendors whose APIs may transition over time for greater "MPI friendliness."

Additionally, we are also considering support for other networking libraries, such as UCX, into ExaMPI. The interactions between the progress of network libraries and ExaMPI's strong progress engine are also of interest. Further, adding support for more profiling tools at all levels is being scoped out.

Future work is motivated by factors impacting applications:

- concurrency of accelerators and MPI requires effective co-scheduling and triggering of messages for effective performance;
- cores appear to be in excess in accelerator systems, suggesting the opportunity for multiple progress engines and shared, concurrent progress engines in modern nodes;
- large-scale codes that use GPUs are spending more of their time percentage-wise in MPI[6], as the GPU math changes the effective granularity of nodes; this increases the importance of overlap as well.
- The interoperability of MPI within itself is questionable when considering weak progress and system-V shared memory and represents an ambiguity or possible flaw in the standard[7]. Strong progress resolves such deadlocks [8].
- Multi-parallel models in a user application can easily deadlock under weak progress; strong progress and shared underlying runtimes therefore become important to to coalesced parallel models in a single application.
- Management of all the concurrency between Pthreads, OpenMP, MPI, and the application is needed in highly threaded systems. ExaMPI is well positioned to explore such co-scheduling experiments.

[5]We expect that other MPI implementations will also operate in this manner [21].

[6]Personal communication, Olga Pearce, LLNL.

[7]On-going discussion in the MPI Forum by Dan Holmes and Rolf Rabenseifner.

[8]At a higher level, [31] indicate ways MPI's specification and implementations can be potentially be streamlined through a strong progress semantic.



Authorized licensed use limited to: Technische Universitaet Muenchen. Downloaded on May 01,2026 at 17:35:55 UTC from IEEE Xplore. Restrictions apply.

Lastly, as Dimitrov [10] noted 20 years ago, time to solution of applications is a superior test of MPI goodness than is zero-message ($\frac{1}{2}$-RTT) latency when grading MPI performance. Thus, a major effort in the coming years is to measure application performance with ExaMPI leveraging strong progress vs. other implementations and configurations on modern architectures.

## VII. Conclusion

In this paper, we presented updates to our latest version of our research MPI implementation, ExaMPI. Along with a recap of the motivation for ExaMPI, APIs and features new to ExaMPI were described. Notable improvements include support for a Libfabric-based network transport and the support for all collective functions (including the persistent versions introduced in MPI 4.0). We also tested the latency of ExaMPI and three production MPI implementations on the Epyc cluster at the University of Tennessee at Chattanooga. The results show that that ExaMPI has a higher latency at smaller message sizes, but closes the gap once the message sizes are large enough. For ExaMPI's weak progress mode, the latency gap is even smaller. We also conducted an overlap experiment, and found that ExaMPI's strong progress engine is able to achieve decent overlap of communication and computation without the need for extra `MPI_Test` calls. ExaMPI's weak progress engine, did require the use `MPI_Test` calls to achieve overlap, as expected.

## Acknowledgment

This work was supported in part by the U.S. Department of Energy's National Nuclear Security Administration (NNSA) under the Predictive Science Academic Alliance Program (PSAAP-III), Award #DE-NA0003966. This work was performed with partial support from the National Science Foundation under Grants Nos. 1925603, 1918987, 1822191, 1821431, 1925598.

We acknowledge the collaboration of Ignacio Laguna and Kathryn Mohror, Lawrence Livermore National Laboratory. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the authors and do not necessarily reflect the views of the National Science Foundation, the U.S. Department of Energy's National Nuclear Security Administration, or the Sandia National Laboratories.

## References

[1] A. Skjellum, M. Rüfenacht, N. Sultana, D. Schafer, I. Laguna, and K. Mohror, "ExaMPI: A Modern Design and Implementation to Accelerate Message Passing Interface Innovation," in *High Performance Computing*, J. L. Crespo-Mariño and E. Meneses-Rojas, Eds. Cham: Springer International Publishing, 2020, pp. 153–169.

[2] W. Gropp, "MPICH2: A new start for MPI implementations," in *EuroPVM/MPI 2002*, 2002.

[3] R. L. Graham, , G. M. Shipman, B. W. Barrett, R. H. Castain, G. Bosilca, and A. Lumsdaine, "Open MPI: A high-performance, heterogeneous MPI," in *Cluster 2006*, Sep. 2006, pp. 1–9.

[4] "Cray MPI." [Online]. Available: https://pubs.cray.com/content/S-2529/17.05/xctm-series-programming-environment-user-guide-1705-s-2529/mpt

[5] "Intel MPI library." [Online]. Available: https://software.intel.com/en-us/mpi-library

[6] "IBM Spectrum MPI." [Online]. Available: https://tinyurl.com/yy9cwm4p

[7] D. K. Panda, K. Tomko, K. Schulz, and A. Majumdar, "The MVAPICH project: Evolution and sustainability of an open source production quality MPI library for HPC," in *WSPPE*, 2013.

[8] MPI Forum, "MPI: A Message-Passing Interface Standard. 2020 Draft Specification," Univ. of Tennessee, Knoxville, TN, USA, Tech. Rep., 2020, Note: This is a MPI-4 Draft Specification.

[9] A. Ruhela, H. Subramoni, S. Chakraborty, M. Bayatpour, P. Kousha, and D. K. Panda, "Efficient asynchronous communication progress for mpi without dedicated resources," in *Proceedings of the 25th European MPI Users' Group Meeting*, ser. EuroMPI'18. New York, NY, USA: Association for Computing Machinery, 2018. [Online]. Available: https://doi.org/10.1145/3236367.3236376

[10] R. P. Dimitrov, "Overlapping of communication and computation and early binding: Fundamental mechanisms for improving parallel performance on clusters of workstations," Ph.D. dissertation, Mississippi State University, Mississippi State, MS, USA, 2001.

[11] D. Schafer, S. Ghafoor, D. Holmes, M. Ruefenacht, and A. Skjellum, "User-level scheduled communications for mpi," in *2019 IEEE 26th International Conference on High Performance Computing, Data, and Analytics (HiPC)*, 2019, pp. 290–300.

[12] "Exascale Computing Project Proxy Apps Suite," https://proxyapps.exascaleproject.org/ecp-proxy-apps-suite/, september, 2021.

[13] N. Sultana, A. Skjellum, P. Bangalore, I. Laguna, and K. Mohror, "Understanding the usage of mpi in exascale proxy applications," *ExaMPI 2018 Workshop at SC'18*, pp. 1–6, 2018.

[14] I. Laguna, R. Marshall, K. Mohror, M. Ruefenacht, A. Skjellum, and N. Sultana, "A large-scale study of MPI usage in open-source HPC applications," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, ser. SC '19. New York, NY, USA: Association for Computing Machinery, 2019. [Online]. Available: https://doi.org/10.1145/3295500.3356176

[15] "Openblas." [Online]. Available: https://github.com/xianyi/OpenBLAS

[16] A. Hassani, "Toward a scalable, transactional, fault-tolerant message passing interface for petascale and exascale machines," Ph.D. dissertation, The University of Alabama at Birmingham, 2016.

[17] N. Sultana, M. Rüfenacht, A. Skjellum, I. Laguna, and K. Mohror, "Failure recovery for bulk synchronous applications with MPI stages," *Parallel Computing*, vol. 84, pp. 1 – 14, 2019. [Online]. Available: http://www.sciencedirect.com/science/article/pii/S0167819118303260

[18] D. Schafer, I. Laguna, A. Skjellum, N. Sultana, and K. Mohror, "Extending the MPI stages model of fault tolerance," in *2020 Workshop on Exascale MPI (ExaMPI)*. IEEE, 2020, pp. 52–61.

[19] A. Skjellum, D. Schafer, and I. Laguna, "MPI sessions and fault tolerance: A position paper," September 2021, accepted to EuroMPI 2021.

[20] T. Herschberg, D. Schafer, and A. Skjellum, "Integrating MPI sessions with topological connection building and collective communication," September 2021, accepted to EuroMPI 2021.

[21] N. Hjelm, H. Pritchard, S. K. Gutiérrez, D. J. Holmes, R. Castain, and A. Skjellum, "MPI Sessions: Evaluation of an Implementation in Open MPI," in *2019 IEEE International Conference on Cluster Computing (CLUSTER)*, 2019, pp. 1–11.

[22] J. Stern, Q. Xiong, J. Sheng, A. Skjellum, and M. Herbordt, "Accelerating MPI_Reduce with FPGAs in the Network," in *Workshop on Exascale MPI*, 2017, https://www.bu.edu/caadlab/exampi17.pdf.

[23] Q. Xiong, A. Skjellum, and M. Herbordt, "Accelerating MPI Message Matching Through FPGA Offload," in *2018 28th International Conference on Field Programmable Logic and Applications (FPL)*, 2018, pp. 191–1914, doi: 10.1109/ FPL.2018.00039.

[24] Q. Xiong, P. Bangalore, A. Skjellum, and M. Herbordt, "MPI Derived Datatypes: Performance and Portability Issues," in *25th European MPI Users' Group Meeting*, 2018, doi: 10.1145/ 3236367.3236378.

[25] Q. Xiong, C. Yang, R. Patel, T. Geng, A. Skjellum, and M. Herbordt, "GhostSZ: A Transparent SZ Lossy Compression Framework with FPGAs," in *2019 IEEE 27th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*, 2019, pp. 258–266, doi: 10.1109/FCCM.2019.00042.

[26] Q. Xiong, "FPGA Acceleration of High Performance Computing Communication Middleware," Ph.D. dissertation, Department of Electrical and Computer Engineering, Boston University, 2019.



Authorized licensed use limited to: Technische Universitaet Muenchen. Downloaded on May 01,2026 at 17:35:55 UTC from IEEE Xplore. Restrictions apply.

[27] Q. Xiong, C. Yang, P. Haghi, A. Skjellum, and M. Herbordt, "Accelerating MPI Collectives with FPGAs in the Network and Novel Communicator Support," in *FCCM*, 2020.
[28] P. Haghi, A. Guo, Q. Xiong, R. Patel, C. Yang, T. Geng, J. T. Broaddus, R. Marshall, A. Skjellum, and M. C. Herbordt, "Fpgas in the network and novel communicator support accelerate mpi collectives," in *2020 IEEE High Performance Extreme Computing Conference (HPEC)*, 2020, pp. 1–10.
[29] P. Haghi, A. Guo, T. Geng, A. Skjellum, and M. Herbordt, "Workload Imbalance in HPC Applications: Effect on Performance of In-Network Processing," in *HPExC*, 2021.
[30] A. Worley, P. Soundararajan, D. Schafer, R. E. Grant, M. G. Dosanjh, P. V. Bangalore, A. Skjellum, and S. Ghafoor, "Design of a Portable Implementation of Partitioned Point-to-Point Communication Primitives," in *50th International Conference on Parallel Processing - ICPP: Workshops*, ser. Fourteenth International Workshop on Parallel Programming Models and Systems Software for High-End Computing (P2S2). New York, NY, USA: Association for Computing Machinery, 2021.
[31] D. J. Holmes, A. Skjellum, and D. Schafer, "Why is MPI (perceived to be) so complex? part 1—does strong progress simplify MPI?" in *27th European MPI Users' Group Meeting*, 2020, pp. 21–30.



Authorized licensed use limited to: Technische Universitaet Muenchen. Downloaded on May 01,2026 at 17:35:55 UTC from IEEE Xplore. Restrictions apply.